



Sri Lanka Institute of Information Technology

Assignment 1

Data Warehouse & Business Intelligence

Submitted by:

K.S. Diyana - IT23712836

Table of Contents

Task 1: Data set selection.....3

Task 2: Preparation of data sources5

Task 3: Solution architecture8

Task 4: Data warehouse design & development9

Task 5: ETL Development10

Task 6: ETL development – Accumulating fact tables20

Task 1: Data set selection

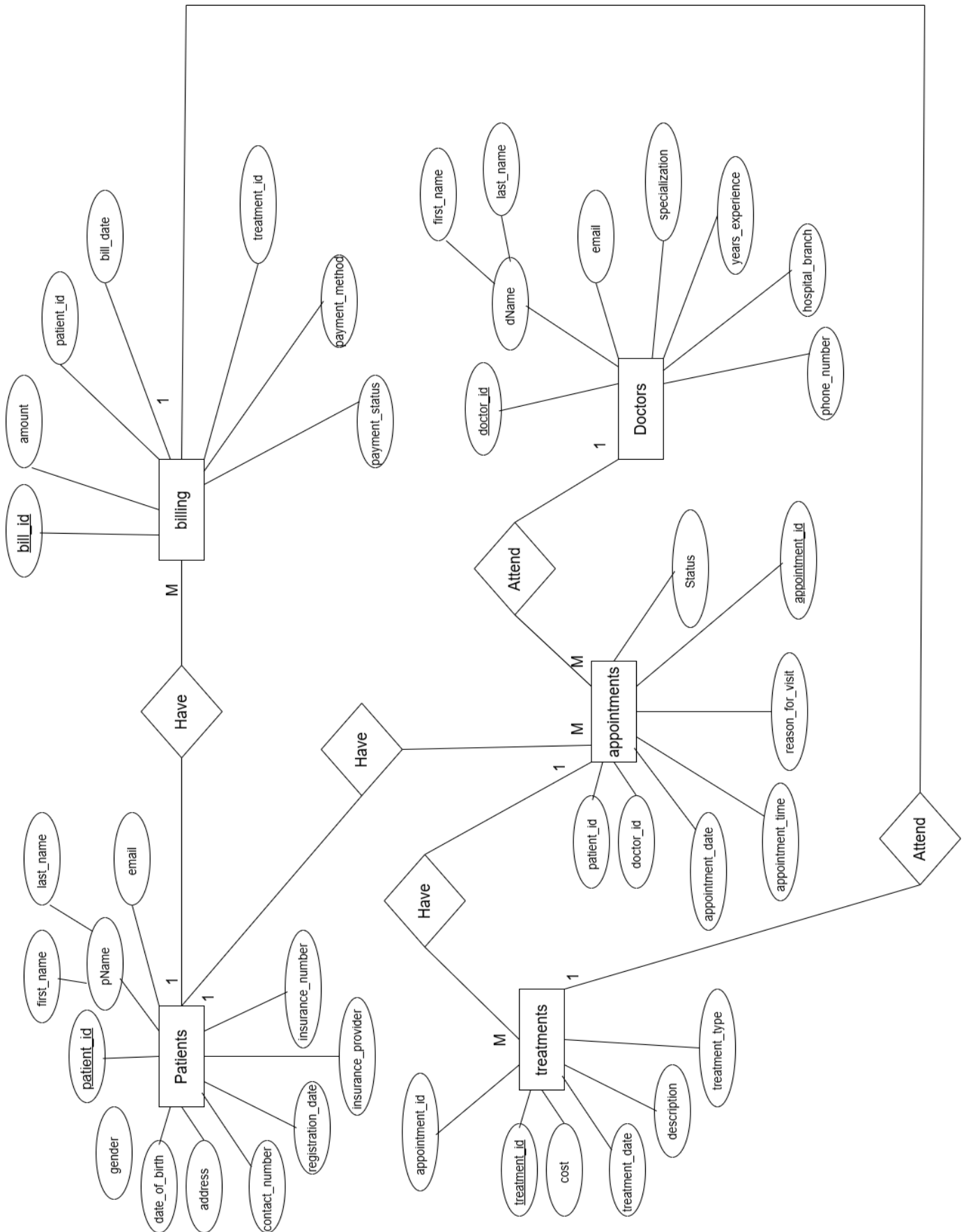
The chosen data source for this assignment is a hospital management system dataset. This dataset is synthetically generated for educational purposes and simulates the operations of a multi-branch hospital over at least one year.

Dataset: <https://www.kaggle.com/datasets/kanakbaghel/hospital-management-dataset>

To meet the assignment criteria of having sufficient records and coverage for one year, additional data was auto-generated based on the existing dataset. The generated records maintain proper relationships between tables and realistic data values for all attributes.

- a) **patients.csv** - Contains patient demographics, contact details, registration dates, and insurance information
- b) **appointments.csv** - Records patient appointments including appointment date, time, reason for visit, attending doctor, and status (Scheduled, Completed, Cancelled)
- c) **treatments.csv** - Details treatments administered during appointments, including type, description, cost, and treatment date
- d) **billing.csv** - Billing information linked to treatments, including total amount, payment method (Cash, Card, Insurance), and payment status.
- e) **doctors.txt** - Contains doctor profiles including ID, name, specialization, years of experience, hospital branch, and contact details.

ER Diagram



Task 2: Preparation of data sources

The final types of data sources are mentioned below

CSV files

- Patients
- Appointments
- Treatments
- Billing

TXT files

- doctors

Patients		Source Type: CSV File
Column Name	Data Type	Description
patient_id	int	Unique identifier for each patient
first_name	varchar(50)	Patient's first name
last_name	varchar(50)	Patient's last name
gender	varchar(10)	Gender of the patient
date_of_birth	date	Patient's date of birth
contact_number	varchar(20)	Phone number of the patient
address	varchar(150)	Residential address of the patient
registration_date	date	Date when the patient registered
insurance_provider	varchar(100)	Name of the insurance company
insurance_number	varchar(50)	Insurance policy number
email	varchar(100)	Email address of the patient

Doctors		Source Type: TXT File
Column Name	Data Type	Description
doctor_id	int	Unique identifier for each doctor
first_name	varchar(50)	Doctor's first name
last_name	varchar(50)	Doctor's last name
specialization	varchar(100)	Medical specialization of the doctor
phone_number	varchar(20)	Contact number of the doctor
years_experience	int	Number of years of experience
hospital_branch	varchar(100)	Branch where the doctor works
email	varchar(100)	Email address of the doctor

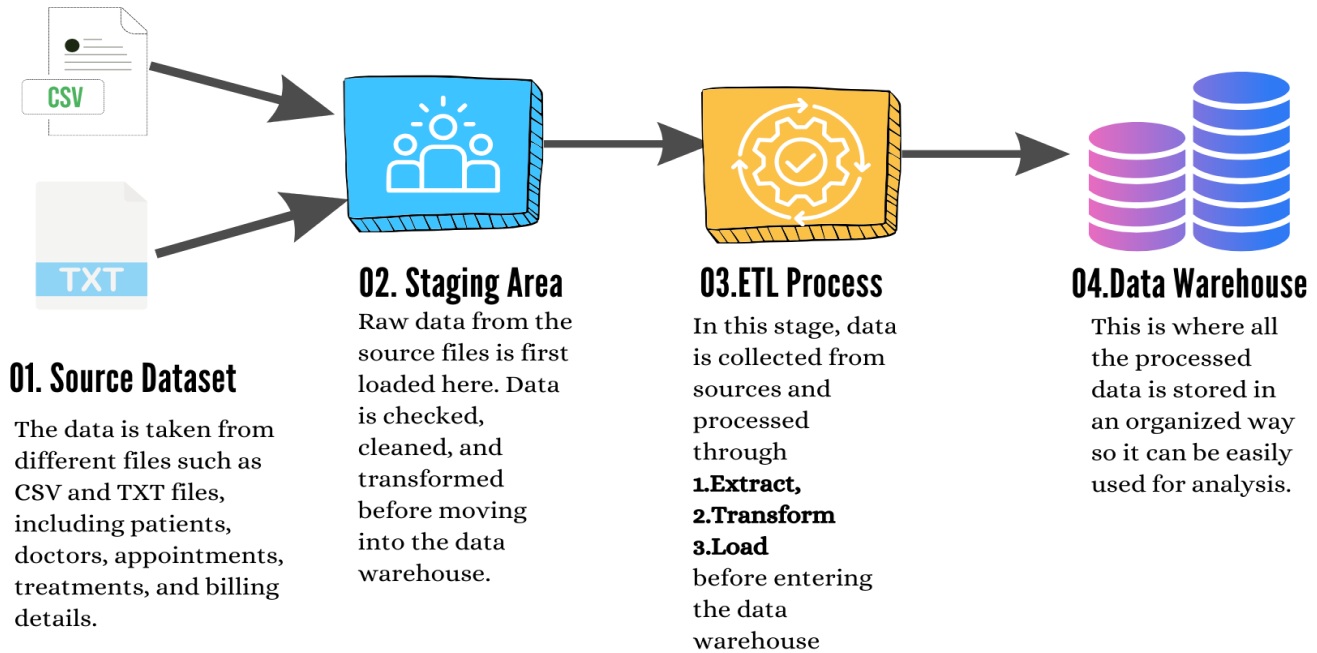
Appointments		Source Type: CSV File
Column Name	Data Type	Description
appointment_id	int	Unique identifier for each appointment
patient_id	int	Reference to the patient
doctor_id	int	Reference to the doctor
appointment_date	date	Date of the appointment
appointment_time	time	Time of the appointment
reason_for_visit	varchar(255)	Reason for the visit
status	varchar(50)	Status of appointment

Treatments		Source Type: CSV File
Column Name	Data Type	Description
treatment_id	int	Unique identifier for each treatment
appointment_id	int	Reference to the appointment
treatment_type	varchar(100)	Type of treatment
description	varchar(255)	Details about the treatment
cost	decimal(10,2)	Cost of the treatment
treatment_date	date	Date when treatment was given

Billing		Source Type: CSV File
Column Name	Data Type	Description
bill_id	int	Unique identifier for each bill
patient_id	int	Reference to the patient
treatment_id	int	Reference to the treatment
bill_date	date	Date of billing
amount	decimal(10,2)	Total bill amount
payment_method	varchar(50)	Method of payment
payment_status	varchar(50)	Status of the payment

Task 3: Solution architecture

ARCHITECTURAL DIAGRAM



Task 4: Data warehouse design & development

Hospital_DW is designed as a star schema with one fact table and five dimension tables, including a Date dimension.

The grain of the FactBilling table is defined as one record per billing transaction per patient per treatment.

Dimension Tables – DimPatient

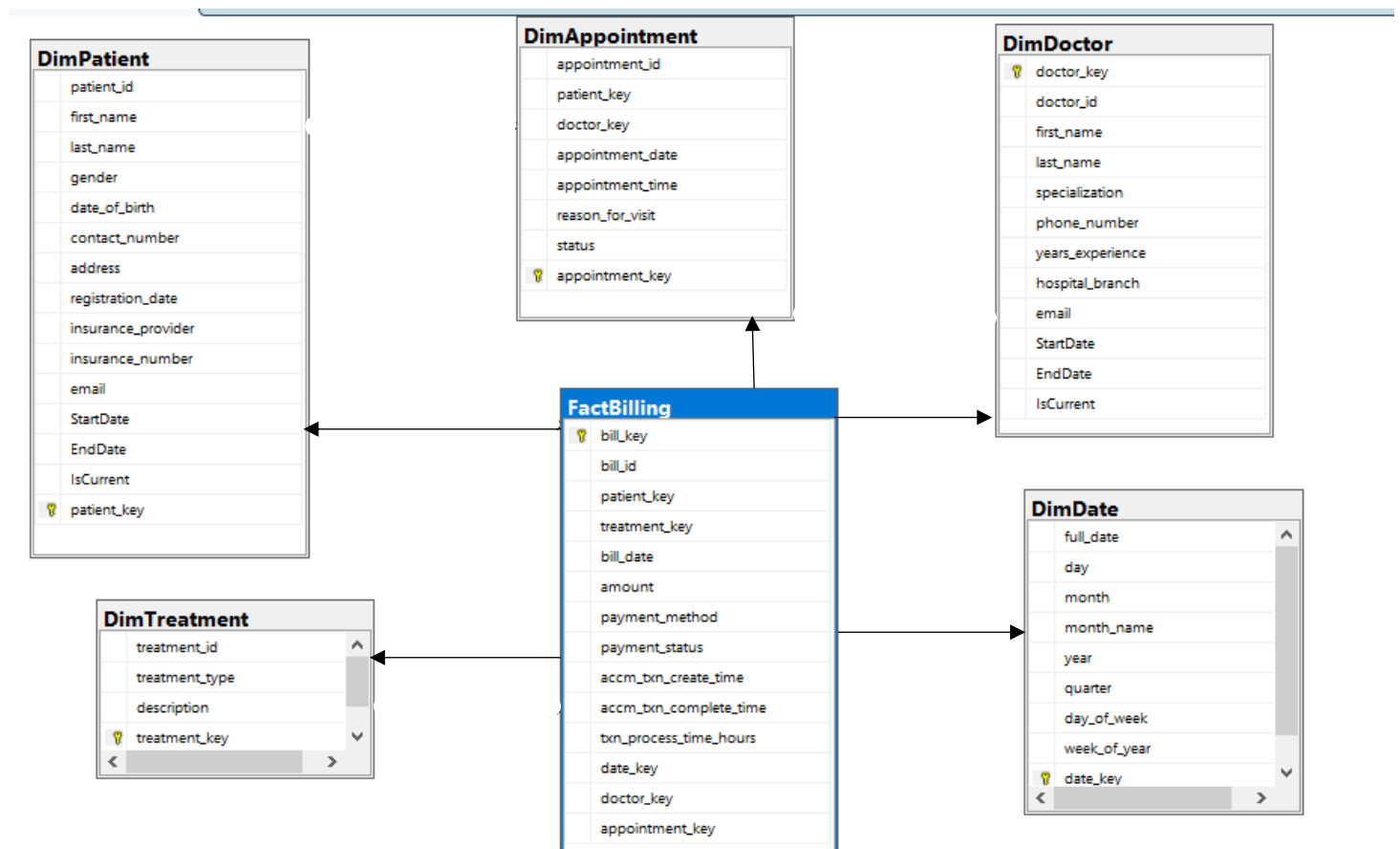
DimDoctor

DimTreatment

DimAppointment

DimDate

Fact Tables - FactBilling



Task 5: ETL Development

Extracting Process

The ETL process uses SSIS in Visual Studio to extract and load data. The system works with two main types of sources: CSV files and TXT.

CSV files : patients , appointments, billing, treatments

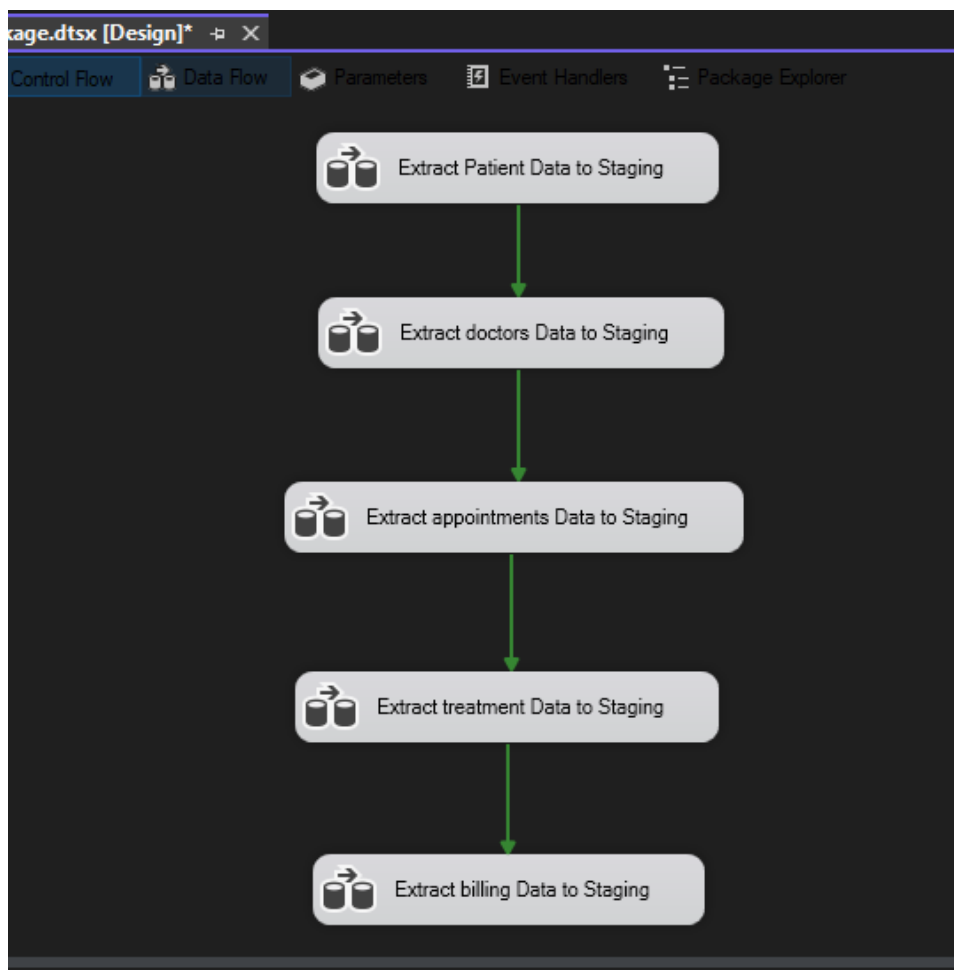
TXT files : doctors

those are read using a Flat File Source, while database sources are handled through OLE DB Source.

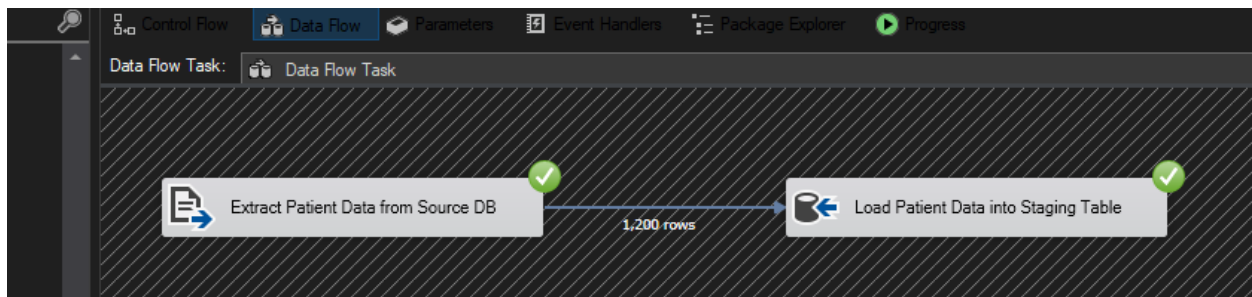
Data from these sources is loaded into staging tables (e.g., Stg_Patients, Stg_Doctors) using OLE DB Destination.

Before loading new data, an Execute SQL Task clears the staging tables so old data is removed and duplicates are avoided when the ETL runs multiple times.

Control Flow



1. Staging Patient Data



Extract and Load Patient Data to Staging

- Source: patients.csv
- Destination: Stg_Patients
- Data conversion applied for address and gender columns

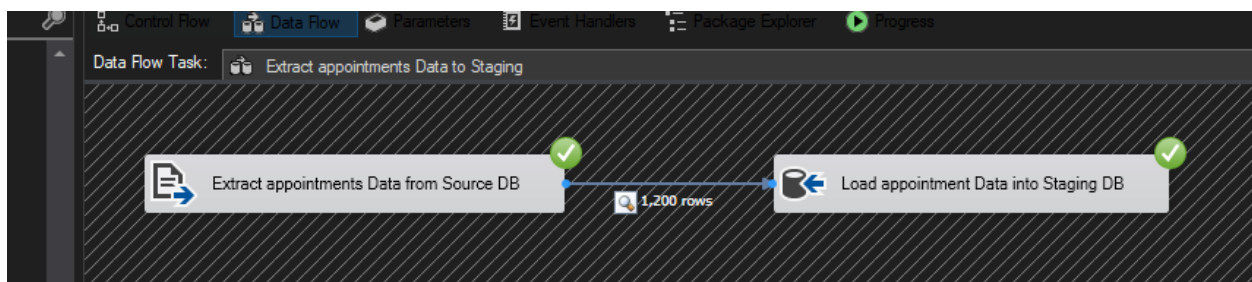
2. Staging Doctor Data



Extract and Load Doctor Data to Staging

- Source: doctors.txt
- Destination: Stg_Doctors

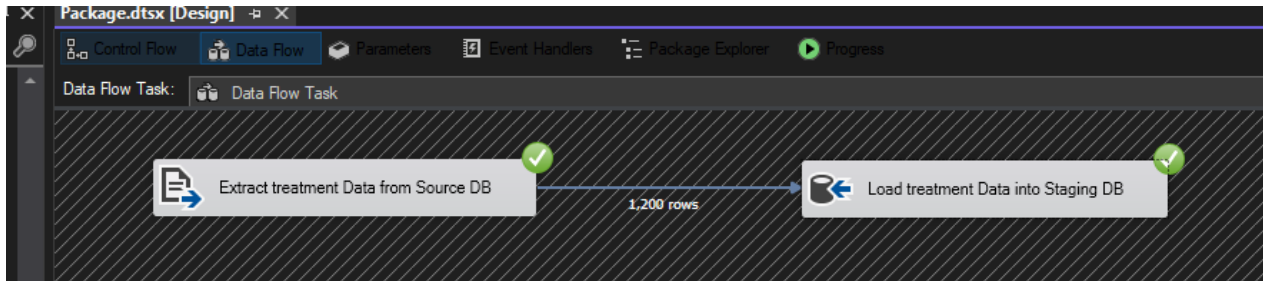
3. Staging Appointment Data



Extract and Load Appointment Data to Staging

- Source: appointments.csv
- Destination: Stg_Appointments
- Date formatting conversion applied

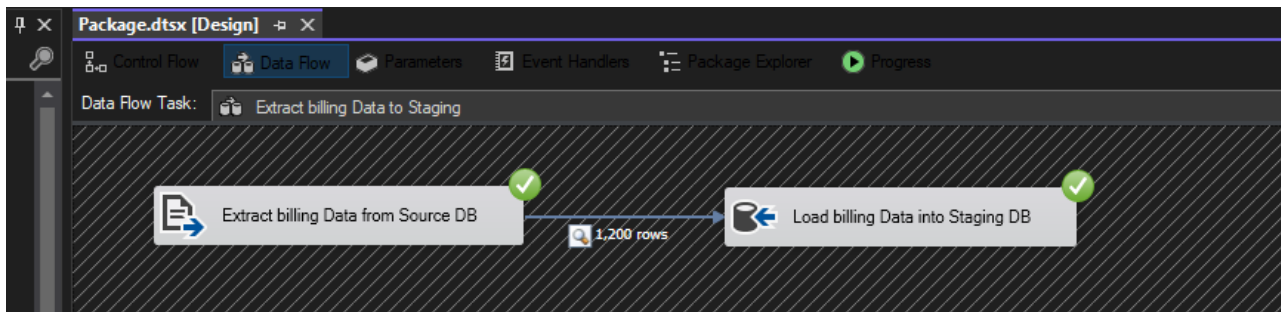
4. Staging Treatment Data



Extract and Load Treatment to Staging

- Source: treatment.csv
- Destination: Stg_treatments
- Date formatting conversion applied

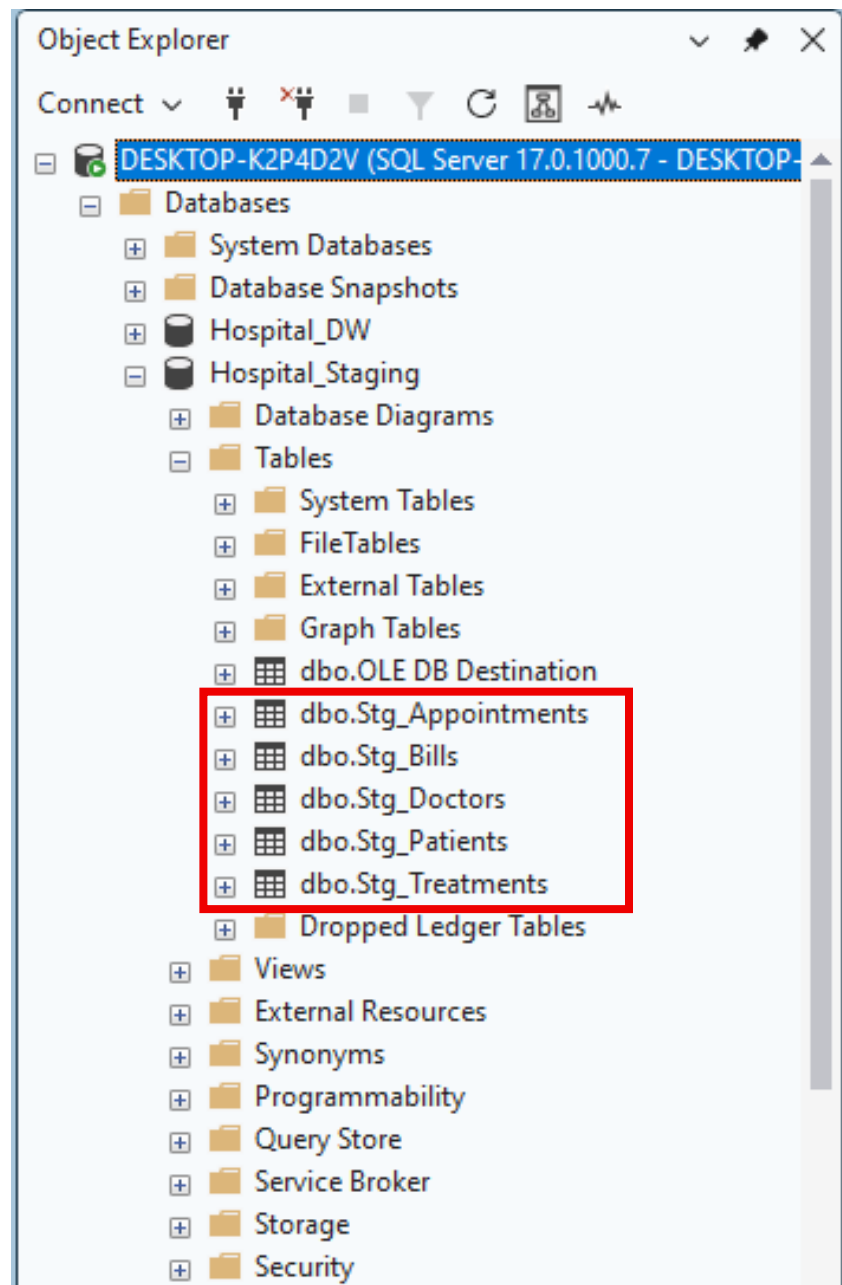
5. Staging Billing Data



Extract and Load Billing Data to Staging

- Source: billing.csv
- Destination: Stg_billing
- Date formatting conversion applied

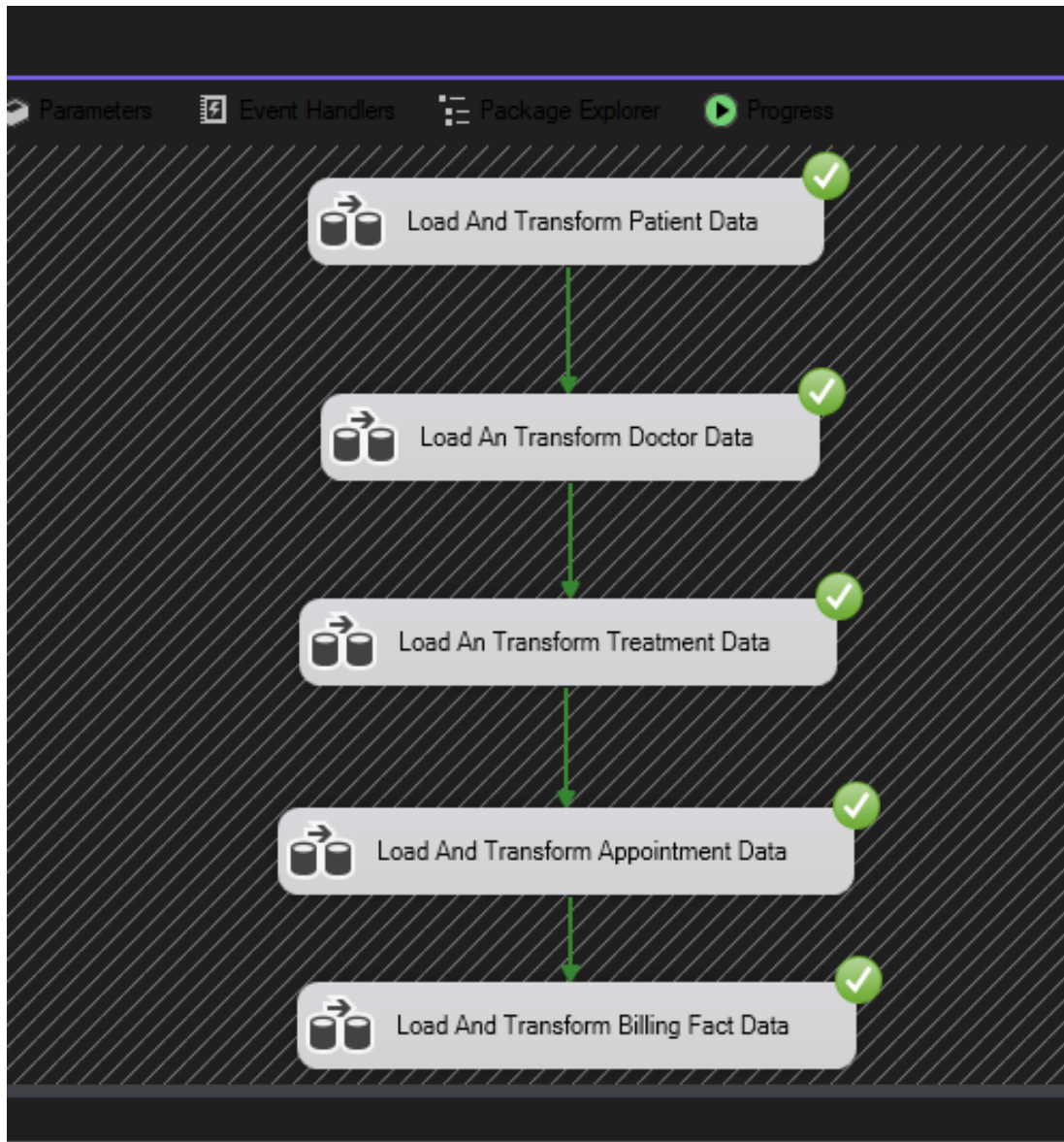
Staging Table Creation



Transformation & Loading

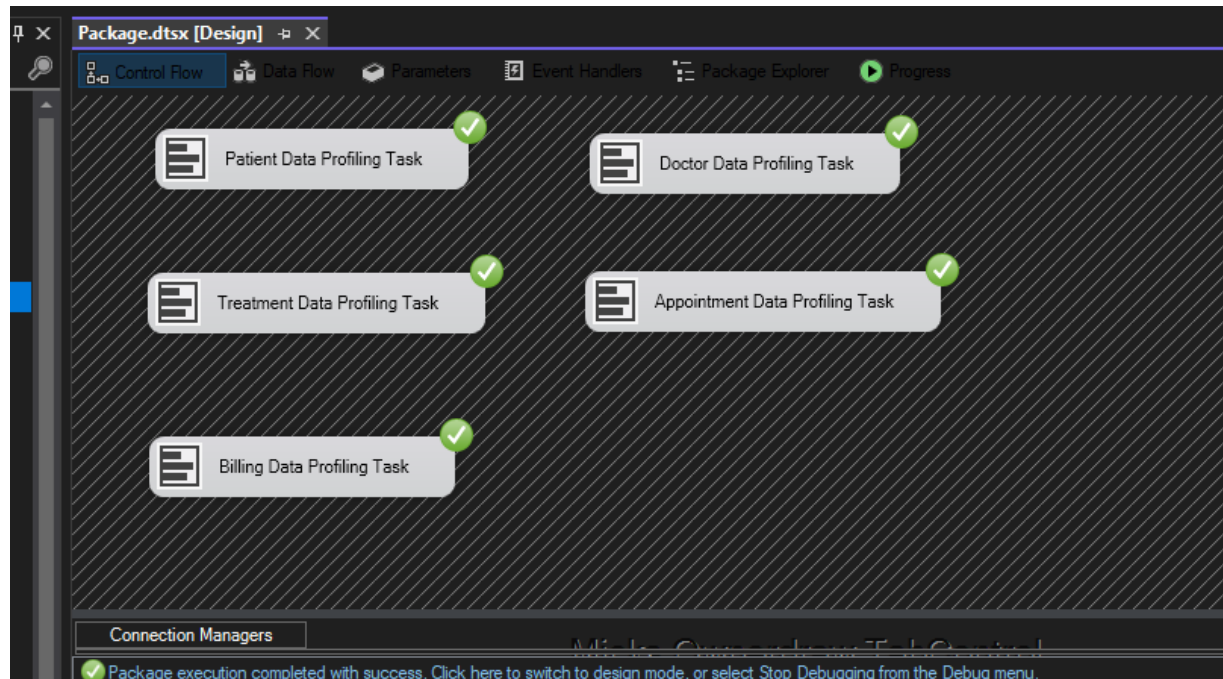
Data is extracted from staging tables, transformed, and loaded into dimension and fact tables for analysis.

Data transformation is developed according to the dimension modelling designed bellow



DATA PROFILING

Data profiling was performed on staging tables to identify data quality issues such as NULL values, duplicate records, and inconsistent formats. SQL queries were used to analyze each dataset. Issues such as missing gender values and duplicate patient IDs were identified.



Every staging table had run a profiling task and saved output in a selected location • By referring these data profiles, the developer is able to identify the issues with staging data

Transform Data - Slowly Changing Dimensions

Slowly Changing Dimensions (SCD) are used to manage changes in dimension tables over time.

In this project, both DimPatient and DimDoctor are implemented using a combination of:

- SCD Type 1 → Overwrite old values
- SCD Type 2 → Maintain historical records

The SCD logic is implemented using the Slowly Changing Dimension Wizard in SSIS.

Patient Dimension

SCD Type 2 (Keep History) - These values change over time and historical tracking is important.

Applied for – address

insurance_provider

SCD Type 1 (Overwrite) - Only the latest value is required.

Applied for – contact_number

Email

Fixed Attributes (No Change Expected)

Applied for – patient_id

gender

date_of_birth

last_name

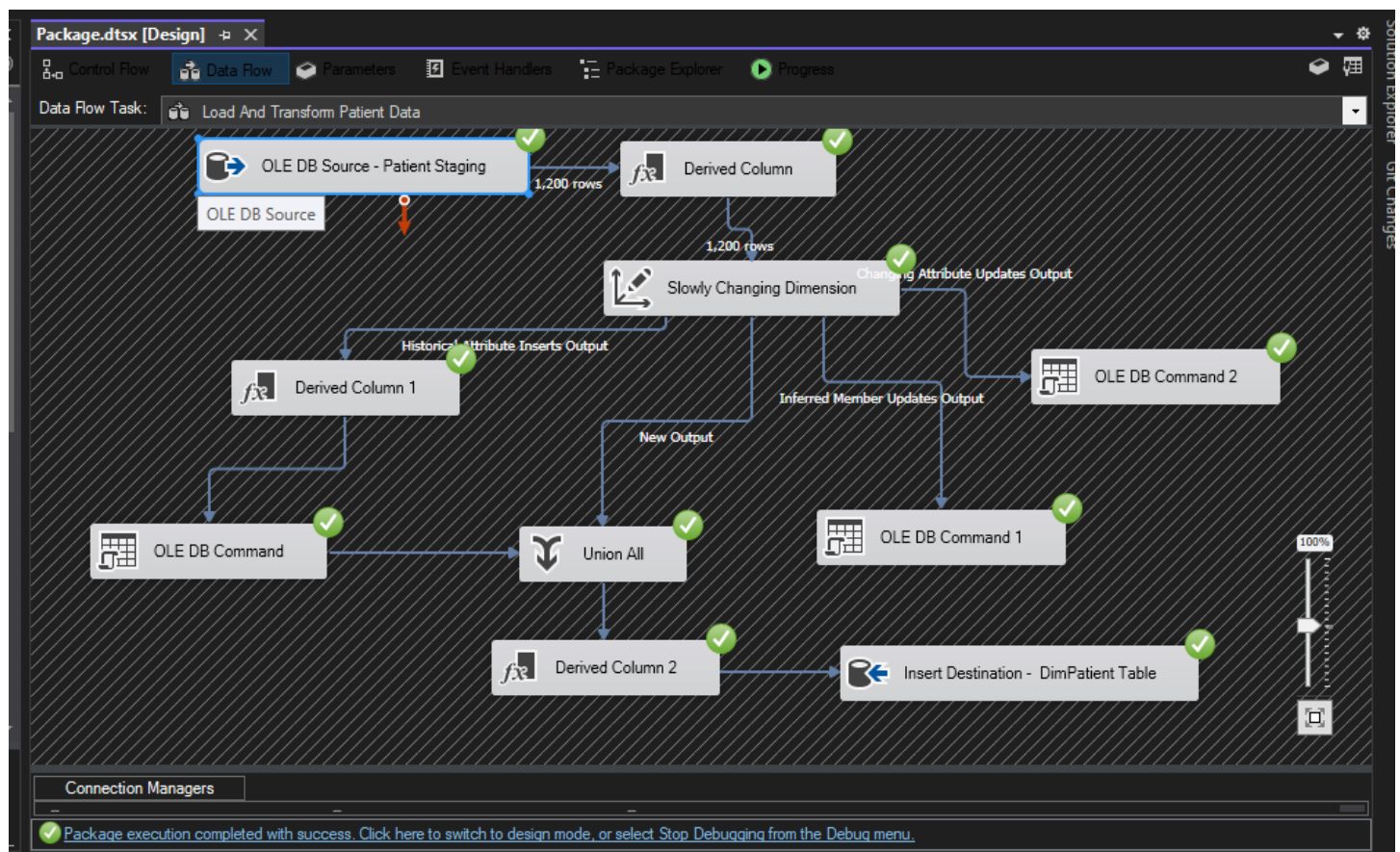
first_name

SCD Supporting Columns

StartDate → Record start date

EndDate → Record expiry date

IsCurrent → Indicates active record (1 = current, 0 = old)



Doctor Dimension

SCD Type 2 (Keep History) - These values change over time and historical tracking is important.

Applied for – specialization

years_experience

hospital_branch

SCD Type 1 (Overwrite) - Only the latest value is required.

Applied for – phone_number

Email

Fixed Attributes (No Change Expected)

Applied for – doctor_id

first_name

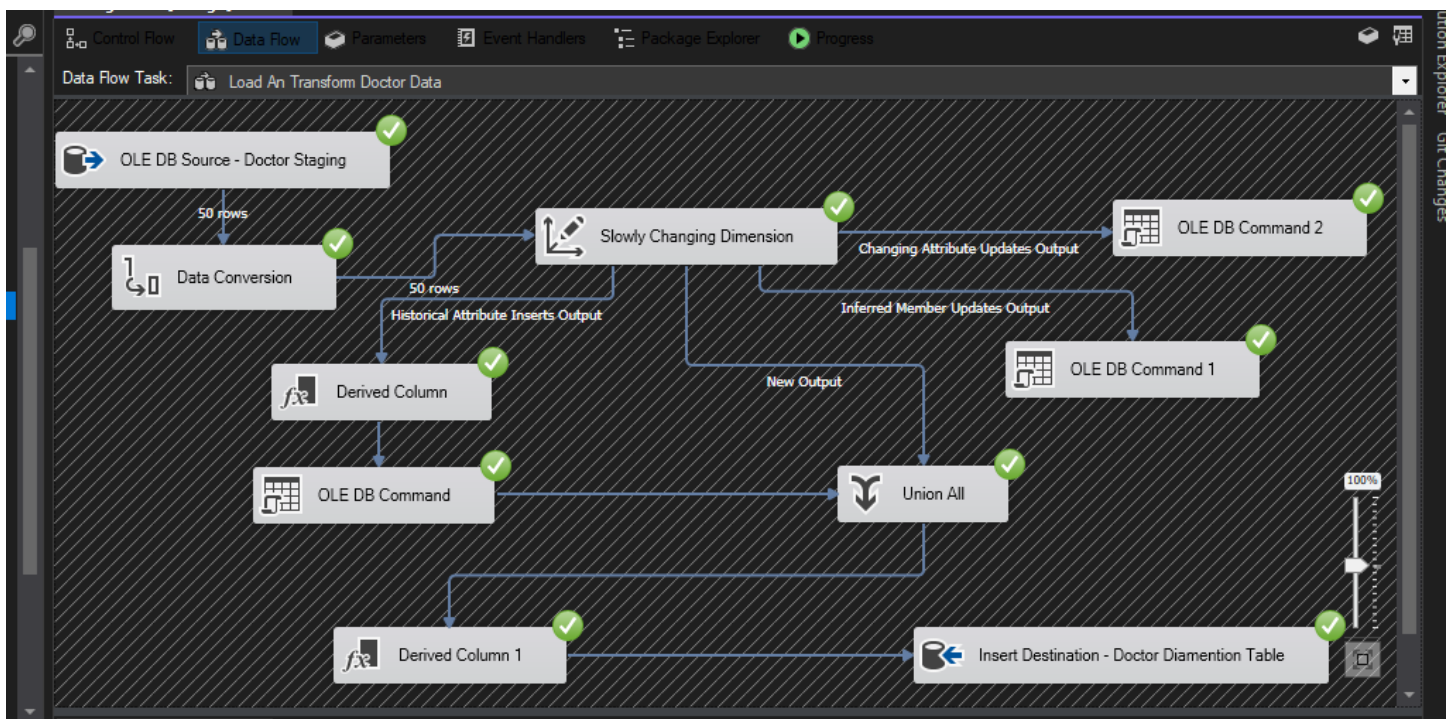
last_name

SCD Supporting Columns

StartDate → Record start date

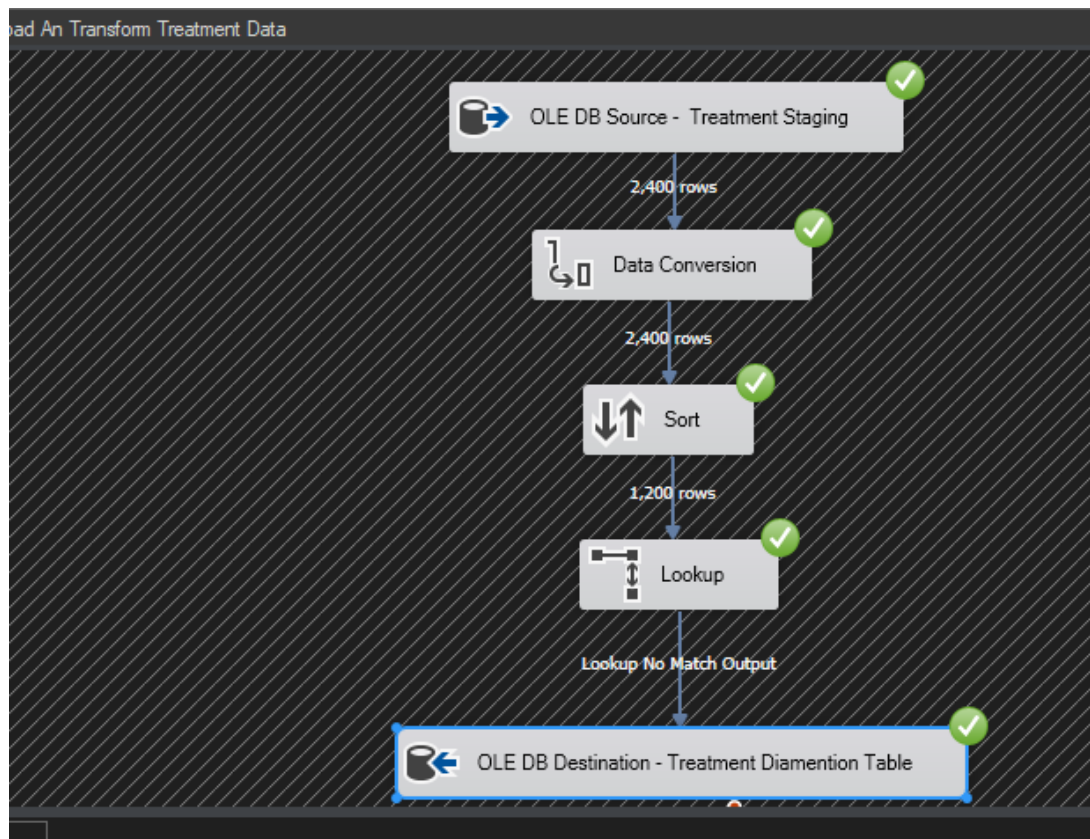
EndDate → Record expiry date

IsCurrent → Indicates active record (1 = current, 0 = old)



DimTreatment

Treatment data is loaded into the dimension table to categorize and describe medical services, enabling analysis based on treatment type and description.



DimAppointment

Appointment data is transformed and loaded into the fact table by linking patient and doctor dimensions to support scheduling and visit analysis.



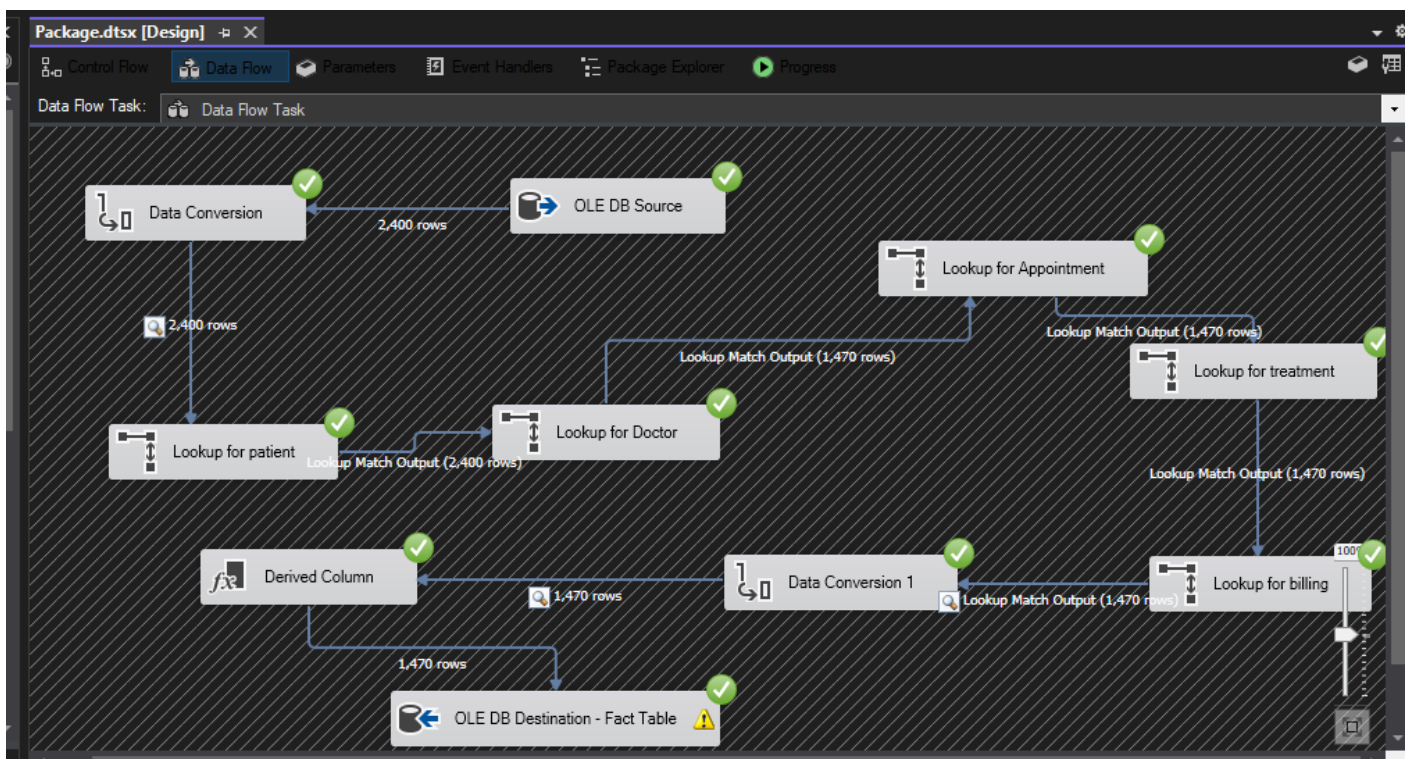
DimDate

Data is generated and loaded into the date dimension table to provide structured date-related attributes (day, month, year, etc.) for efficient time-based analysis.

```
date codes.s...V\user (57))  X  DESKTOP-K2P4... dbo.DimDate  DESKTOP-K2P4...o.FactBilling
1  DECLARE @StartDate DATE = '2020-01-01';
2  DECLARE @EndDate DATE = '2030-12-31';
3
4  WHILE @StartDate <= @EndDate
5  BEGIN
6      INSERT INTO DimDate
7      VALUES (
8          CONVERT(INT, FORMAT(@StartDate, 'yyyyMMdd')),
9          @StartDate,
10         DAY(@StartDate),
11         MONTH(@StartDate),
12         DATENAME(MONTH, @StartDate),
13         YEAR(@StartDate),
14         DATEPART(QUARTER, @StartDate),
15         DATENAME(WEEKDAY, @StartDate),
16         DATEPART(WEEK, @StartDate)
17     );
18     SET @StartDate = DATEADD(DAY, 1, @StartDate);
19
20 END;
```

FactBilling

Billing data from staging is transformed and loaded into the fact table by linking with dimension keys to support transaction-level analysis and reporting.



SQLQuery1.s...V\user (61)*

```
SELECT * FROM FactBilling;
```

100 % No issues found Ln: 5, Ch: 1 TABS CRLF UTF-8

Results Messages

	bill_key	bill_id	patien...	treat...	bill_date	amount	payment_method	payment_status	accm_txn_create_time	accm_txn_complete_time	txn_...	date_key	doctor_key	appointment
1	1	1	1033	543	2025-07-30	11435.00	Card	Pending	2026-03-30 18:47:56.150	2026-03-30 18:47:56.150	0	1	30	673
2	2	2	193	321	2025-07-13	11187.00	Insurance	Paid	2026-03-30 18:47:56.150	2026-03-30 18:47:56.150	0	2	48	224
3	3	3	1109	568	2026-02-03	19922.00	Insurance	Paid	2026-03-30 18:47:56.150	2026-03-30 18:47:56.150	0	3	42	1084
4	4	4	133	101	2026-02-02	39216.00	Insurance	Pending	2026-03-30 18:47:56.150	2026-03-30 18:47:56.150	0	4	2	123
5	5	6	1155	207	2025-10-10	14578.00	Card	Paid	2026-03-30 18:47:56.150	2026-03-30 18:47:56.150	0	6	35	287
6	6	7	448	209	2025-10-04	3789.00	Insurance	Paid	2026-03-30 18:47:56.150	2026-03-30 18:47:56.150	0	7	24	200
7	7	8	694	494	2025-07-19	18328.00	Cash	Paid	2026-03-30 18:47:56.150	2026-03-30 18:47:56.150	0	8	48	133
8	8	12	1069	410	2025-07-10	17879.00	Insurance	Paid	2026-03-30 18:47:56.150	2026-03-30 18:47:56.150	0	12	30	1026
9	9	13	12	997	2025-06-21	22066.00	Card	Pending	2026-03-30 18:47:56.150	2026-03-30 18:47:56.150	0	13	35	285
10	10	15	62	105	2025-04-09	32309.00	Insurance	Pending	2026-03-30 18:47:56.150	2026-03-30 18:47:56.150	0	15	32	594
11	11	17	70	595	2026-03-14	26984.00	Card	Pending	2026-03-30 18:47:56.150	2026-03-30 18:47:56.150	0	17	8	1029
12	12	22	110	806	2025-07-11	8763.00	Card	Paid	2026-03-30 18:47:56.150	2026-03-30 18:47:56.150	0	22	4	905
13	13	23	12	956	2025-08-26	36191.00	Card	Pending	2026-03-30 18:47:56.150	2026-03-30 18:47:56.150	0	23	35	285
14	14	24	639	373	2025-11-02	41819.00	Cash	Paid	2026-03-30 18:47:56.150	2026-03-30 18:47:56.150	0	24	4	401
15	15	25	9	753	2025-10-06	48125.00	Cash	Paid	2026-03-30 18:47:56.150	2026-03-30 18:47:56.150	0	25	46	1079
16	16	26	918	320	2025-05-03	26971.00	Insurance	Paid	2026-03-30 18:47:56.150	2026-03-30 18:47:56.150	0	26	47	291
17	17	27	341	690	2025-10-14	28725.00	Card	Paid	2026-03-30 18:47:56.150	2026-03-30 18:47:56.150	0	27	41	648
18	18	28	711	569	2026-03-17	34481.00	Cash	Failed	2026-03-30 18:47:56.150	2026-03-30 18:47:56.150	0	28	23	915
19	19	29	410	910	2026-02-10	15618.00	Insurance	Failed	2026-03-30 18:47:56.150	2026-03-30 18:47:56.150	0	29	36	973
20	20	31	998	603	2025-07-08	35329.00	Cash	Pending	2026-03-30 18:47:56.150	2026-03-30 18:47:56.150	0	31	14	486
21	21	33	857	81	2025-05-06	12949.00	Cash	Paid	2026-03-30 18:47:56.150	2026-03-30 18:47:56.150	0	33	28	332

Query executed successfully. DESKTOP-K2P4D2V (17.0 RTM) DESKTOP-K2P4D2V\user (61) Hospital_DW 00:00:00 Row: 1, Col: 1 1,470 rows

Task 6: ETL development – Accumulating fact tables

The objective of this task is to implement an **accumulating fact table** to track transaction lifecycle stages. The fact table is extended to store creation time, completion time, and processing duration in hours.

Fact Table Modification

SQLQuery2.s...V\user (68)* DESKTOP-K2P4...Bill_Updates

```

1 Use Hospital_DW
2
3 ALTER TABLE FactBilling
4 ADD accm_txn_create_time DATETIME,
5     accm_txn_complete_time DATETIME,
6     txn_process_time_hours INT;
7
8
9 SELECT COLUMN_NAME
10 FROM Hospital_DW.INFORMATION_SCHEMA.COLUMNS
11 WHERE TABLE_NAME = 'FactBilling';

```

100 % No issues found

Results Messages

	COLUMN_NAME
1	bill_key
2	bill_id
3	patient_key
4	treatment_key
5	bill_date
6	amount
7	payment_method
8	payment_status
9	accm_txn_create_time
10	accm_txn_complete_time
11	txn_process_time_hours

Query executed successfully.

ETL Process for Updating Fact Table

A separate ETL process is developed to update the fact table using the completion dataset.

1. Read transaction update dataset (CSV/Table)
2. Match records using txn_id
3. Update accm_txn_complete_time
4. Calculate processing duration in hours

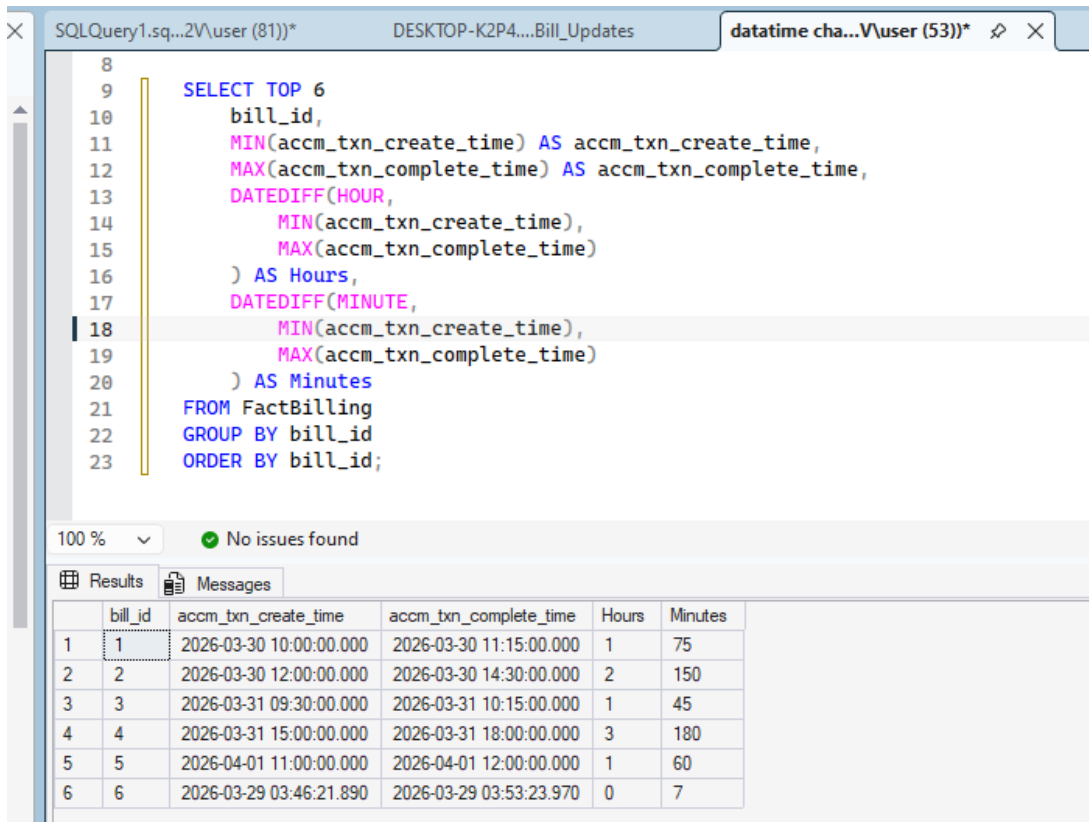


Processing Time Calculation

The transaction processing time is calculated using the difference between completion time and creation time.

$$\text{txn_process_time_minutes} = \text{Completion Time} - \text{Creation Time (in minutes)}$$

Expected Outcome



The screenshot displays a SQL Server Enterprise Manager interface. At the top, there are three tabs: 'SQLQuery1.sql...2V\user (81))*', 'DESKTOP-K2P4....Bill_Updates', and 'datetime cha...V\user (53))*'. The active window shows a SQL query in a text editor. The query is as follows:

```
8
9  SELECT TOP 6
10     bill_id,
11     MIN(accm_txn_create_time) AS accm_txn_create_time,
12     MAX(accm_txn_complete_time) AS accm_txn_complete_time,
13     DATEDIFF(HOUR,
14         MIN(accm_txn_create_time),
15         MAX(accm_txn_complete_time)
16     ) AS Hours,
17     DATEDIFF(MINUTE,
18         MIN(accm_txn_create_time),
19         MAX(accm_txn_complete_time)
20     ) AS Minutes
21  FROM FactBilling
22  GROUP BY bill_id
23  ORDER BY bill_id;
```

Below the query editor, the 'Results' tab is active, showing a table with 6 rows and 6 columns. The columns are 'bill_id', 'accm_txn_create_time', 'accm_txn_complete_time', 'Hours', and 'Minutes'. The data is as follows:

	bill_id	accm_txn_create_time	accm_txn_complete_time	Hours	Minutes
1	1	2026-03-30 10:00:00.000	2026-03-30 11:15:00.000	1	75
2	2	2026-03-30 12:00:00.000	2026-03-30 14:30:00.000	2	150
3	3	2026-03-31 09:30:00.000	2026-03-31 10:15:00.000	1	45
4	4	2026-03-31 15:00:00.000	2026-03-31 18:00:00.000	3	180
5	5	2026-04-01 11:00:00.000	2026-04-01 12:00:00.000	1	60
6	6	2026-03-29 03:46:21.890	2026-03-29 03:53:23.970	0	7

After executing the ETL process:

- ✓ Each transaction will have a creation timestamp
- ✓ Completion time will be updated separately
- ✓ Processing time will be automatically calculated
- ✓ The fact table will represent full transaction lifecycle tracking

ERROR HANDLING

Basic error handling was implemented in the ETL process using SSIS logging and error output redirection. Data flow components were configured to redirect rows with errors

e.g., data type mismatches or missing values

to separate outputs for further analysis. Additionally, SSIS logging was enabled to capture execution details, allowing easier identification and troubleshooting of failures during package execution. Error outputs in the data flow (red arrow paths) ensure that invalid records do not interrupt the ETL process.